- *The target index can never have fewer primary shards than the source's index has.* Remember, splitting allows more room for the index.
- *The target index's node must have adequate space.* Make sure the shards are allocated with the appropriate space.

During the splitting operation, all the configurations (settings, mappings, and aliases) are copied from the source index into the newly created target index. Elasticsearch then moves the source segment's hard links to the target index. Finally, all the documents are rehashed, because the documents' home has changed.

Target's index number must be multiple of source's index number

The target index's number of primary shards must be a multiple of the number of primary shards of the source index. To see what happens, provide a non-multiple number to reset `index.number_of_shards` (to `14`: a non-multiple of source's primary shards number, which is 3) as the following query shows:

```
POST all_cars/_split/all_cars_new
{
  "settings": {
    "index.number_of_shards": 14
  }
}
```

Unfortunately, this query throws an exception because we violated the second rule in the previous list. Here's the exception:

```
"reason" : "the number of source shards [3] must be a factor of [14]"
```

Splitting indexes also helps resize a cluster by adding primaries to the original count. The configurations are copied across the source to the target shards. So, other than adding more shards, the split API can't change any settings on the target index. If we just need to increase the number of shards so the data is spread across the newly created indexes, splitting is the best way to go. Also, remember that target indexes must not exist before invoking a splitting operation. Now that we've worked on splitting indexes, let's look at another operation: shrinking an index.

6.8.2 Shrinking an index

While splitting an index expands it by adding additional shards for more space, shrinking is the opposite: it reduces the number of shards. Shrinking helps consolidate all the documents spread over various shards into fewer shards. Doing so can help in use cases like these:

- We had one or move indexes during a holiday period, with data spread across numerous shards. Now that the holiday period is over, we wish to reduce the shard count.
- To increase read speed (search throughput), we added data nodes. Once demand subsides, there is no reason to keep all those nodes active.